SUMÁRIO

1 **INTRODUÇÃO** 2

1.1 **O problema da ordenação** 2

2 **PRELIMINARES** 3

2.1 **A importância da ordenação** 3

2.2 **Ordenação interna e externa** 3

2.3 **Ordenação estável** 4

2.4 **Análise de algoritmos** 4

2.4.1 ***Notação assintótica*** 4

2.4.2 ***Cota inferior*** 5

2.4.3 ***Divisão e conquista e Teorema mestre*** 5

2.4.4 ***Probabilidade e valor esperado*** 6

2.5 **Arquitetura de computadores** 7

2.5.1 ***Hierarquia de memória*** 7

2.5.2 ***Localidade de referência*** 7

3 **ANÁLISE EMPÍRICA** 8

3.1 **Métodos inferiores** 8

REFERÊNCIAS 10

1 INTRODUÇÃO

1.1 O problema da ordenação

O problema da ordenação consiste em reorganizar uma sequência de dados para que seus elementos estejam dispostos em uma ordem específica, geralmente crescente ou decrescente. Em termos formais, o problema pode ser definido da seguinte maneira: dada uma sequência de n números $\langle a_1, a_2, ..., a_n \rangle$, produzir uma permutação $\langle a'_1, a'_2, ..., a'_n \rangle$ da sequência de entrada, tal que $a'_1 \leq a'_2 \leq ... \leq a'_n$ (CORMEN *et al.*, 2009, p. 5).

- Falar sobre o que SERÁ feito neste trabalho

2 PRELIMINARES

2.1 A importância da ordenação

A ordenação é uma das operações fundamentais em computação, com aplicações diretas em diversas áreas, como pesquisa, organização de dados, algoritmos de busca e análise estatística. Além disso, é uma etapa essencial em muitos outros algoritmos, como aqueles utilizados em análise de grafos, aprendizado de máquina e simulação de sistemas.

A importância do problema da ordenação também está relacionada à sua presença em problemas cotidianos, como organização de listas de compras, ordenação de arquivos em sistemas operacionais, ordenação de registros em bancos de dados e priorização de tarefas em filas de execução. Além disso, a resolução desse problema proporciona uma base teórica para o estudo de outros problemas mais complexos em computação.

Compreender e resolver eficientemente o problema da ordenação é crucial para otimizar o desempenho de sistemas computacionais. A escolha de um algoritmo adequado para um determinado cenário pode impactar significativamente os requisitos de tempo e espaço, especialmente em aplicações que envolvem grandes volumes de dados. Assim, o estudo da ordenação não só permite avanços na própria área de computação, mas também contribui para soluções mais eficientes em diversas áreas do conhecimento.

2.2 Ordenação interna e externa

A ordenação pode ser classificada em dois tipos principais: interna e externa, dependendo da forma como os dados são manipulados durante o processo de ordenação.

Ordenação interna ocorre quando todos os dados a serem ordenados cabem na memória principal do computador (RAM). Esse tipo de ordenação é amplamente utilizado em cenários onde o volume de dados é relativamente pequeno e o acesso rápido à memória permite um desempenho eficiente. Exemplos de algoritmos de ordenação interna incluem Bubble Sort, Merge Sort e Quick Sort.

Ordenação externa, por outro lado, é aplicada quando o volume de dados é tão grande que excede a capacidade da memória principal, necessitando do uso de memória secundária, como discos rígidos ou SSDs. Nesse caso, os algoritmos devem ser projetados para minimizar o número de acessos à memória secundária, que é consideravelmente mais lenta do que a memória principal. Exemplos incluem algoritmos de ordenação por blocos

e Merge Sort externo.

Neste trabalho, o foco será exclusivamente na ordenação interna, abordando os principais algoritmos utilizados, suas características e desempenhos em diferentes cenários.

2.3 Ordenação estável

A ordenação é considerada estável quando preserva a ordem relativa dos elementos com chaves iguais na sequência de entrada. Em outras palavras, se dois elementos possuem a mesma chave e um deles aparece antes do outro na sequência original, um algoritmo de ordenação estável garantirá que essa ordem relativa seja mantida na sequência ordenada.

A estabilidade em algoritmos de ordenação é especialmente importante em cenários onde os elementos possuem atributos adicionais além da chave utilizada para a ordenação. Por exemplo, em um sistema de gerenciamento de registros, onde cada registro é identificado por um nome e uma data, a estabilidade permite ordenar os registros primeiro por nome e, em seguida, por data, sem perder a ordem relativa estabelecida na primeira ordenação.

A tabela 1 apresenta os algoritmos que serão implementados neste trabalho e indica quais deles são estáveis:

Tabela 1 – Da estabilidade dos algoritmos apresentados

Estável	Não estável
Bolha	Seleção
Coquetel	Quicksort
Inserção	Heapsort
Mergesort	Bucketsort
Countingsort	Shellsort
Radixsort	

2.4 Análise de algoritmos

2.4.1 Notação assintótica

A Notação Assintótica é um conjunto de ferramentas matemáticas utilizada na análise de algoritmos para descrever o comportamento limitante ou de crescimento de uma função à medida que o tamanho da entrada se aproxima do infinito. Seu propósito

principal é fornecer uma forma de classificar e comparar a eficiência de algoritmos, focando apenas na ordem de magnitude do tempo ou espaço consumido e ignorando constantes de proporcionalidade e termos de ordem inferior. Ao expressar a complexidade de tempo de um algoritmo, a notação assintótica permite prever como o algoritmo escalará em grandes volumes de dados, independentemente de detalhes de implementação ou da velocidade específica do hardware.

Definir a complexidade de um algoritmo exige o uso de notações assintóticas precisas para delimitar seu comportamento. A notação $\mathcal{O}(f(n))$ fornece um limite superior para complexidade de um algoritmo, seja ela temporal ou espacial, descrevendo o pior caso. Ela significa que, para valores grandes de entrada n, a complexidade nunca será pior do que uma função $c \cdot f(n)$, onde c é uma constante positiva e $f(n)$ é a função de crescimento. Já a notação $\Theta(f(n))$ fornece um limite assintótico apertado, definindo o comportamento exato da complexidade de um algoritmo. Isso significa que, para n grande, a complexidade está limitado superior e inferiormente por múltiplos constantes de $f(n)$, indicando que $f(n)$ é a ordem de crescimento precisa, tanto no melhor quanto no pior caso.

2.4.2 Cota inferior

Cota inferior define o limite mínimo de desempenho necessário para resolver um problema. Isso significa que nenhum algoritmo pode resolver o problema em menos tempo do que a cota inferior definida, independentemente da abordagem utilizada. A notação usada para descrever a cota inferior é a notação $\Omega(f(n))$.

Para algoritmos de ordenação baseados em comparação, a cota inferior é $\Omega(n \log_2 n)$. Esse resultado demonstra que qualquer algoritmo de ordenação que utilize apenas comparações entre elementos, no pior caso, precisará de pelo menos $n \log_2 n$ comparações para ordenar n elementos. Este limite foi provado por (KNUTH, 1998), onde ele utiliza conceitos de árvores de decisão para demonstrar que a profundidade mínima da árvore que representa todas as permutações possíveis dos elementos é $\log_2(n!)$, o que se aproxima de $n \log_2 n$ pela fórmula de Stirling.

2.4.3 Divisão e conquista e Teorema mestre

O paradigma Divisão e conquista é uma das estratégias mais poderosas no projeto de algoritmos, na qual um problema complexo é resolvido de forma recursiva por meio de

três etapas: dividir, conquistar e combinar. Na etapa de divisão, o problema original de tamanho n é quebrado em a subproblemas menores, cada um de tamanho n/b. Na etapa de conquista, os subproblemas são resolvidos recursivamente. Finalmente, na etapa de combinação, as soluções dos subproblemas são reunidas para formar a solução final do problema original.

A eficiência de um algoritmo que segue o paradigma Divisão e conquista é caracterizada por uma relação de recorrência da forma $T(n) = aT(n/b) + f(n)$, sendo que $T(n)$ representa o tempo total de execução. O Teorema Mestre é a ferramenta matemática que permite solucionar essas recorrências e determinar sua complexidade assintótica. Ele funciona comparando o custo do trabalho de divisão e combinação, $f(n)$, com o custo do trabalho realizado nas folhas da árvore de recursão, dado por $n^{\log_b a}$. O teorema afirma que a solução $T(n)$ cairá em um de três casos:

- Se $f(n)$ for assintoticamente menor que $n^{\log_b a}$, a complexidade será $O(n^{\log_b a})$ (dominada pelas folhas);
- Se $f(n)$ for assintoticamente maior, a complexidade será $O(f(n))$ (dominada pela raiz);
- E se forem assintoticamente iguais, a complexidade será $O(n^{\log_b a} \log n)$ (equilibrada).

2.4.4 Probabilidade e valor esperado

A probabilidade é a medida numérica da chance de um evento específico ocorrer, variando de zero (impossibilidade) a um (certeza), e é fundamental para modelar a incerteza. Já o valor esperado (ou esperança matemática) é uma medida que resume o resultado mais provável ou a média de longo prazo de um fenômeno aleatório. Ele é calculado ponderando-se cada resultado possível pelo seu respectivo valor de probabilidade, oferecendo uma estimativa útil para a tomada de decisões em cenários de risco, como jogos de azar ou análises financeiras.

Esses dois conceitos matemáticos serão utilizados no próximo capítulo para a análise de complexidade da versão probabilística do algoritmo Quicksort.

2.5 Arquitetura de computadores

A análise de complexidade $\mathcal{O}(f(n))$ é fundamental, mas não é o único fator que determina o tempo de execução real. Em sistemas de computação modernos, a hierarquia de memória e o conceito de localidade de referência também ajudam a entender o desempenho.

2.5.1 Hierarquia de memória

Nulla non mauris vitae wisi posuere convallis. Sed eu nulla nec eros scelerisque pharetra. Nullam varius. Etiam dignissim elementum metus. Vestibulum faucibus, metus sit amet mattis rhoncus, sapien dui laoreet odio, nec ultricies nibh augue a enim. Fusce in ligula. Quisque at magna et nulla commodo consequat. Proin accumsan imperdiet sem. Nunc porta. Donec feugiat mi at justo. Phasellus facilisis ipsum quis ante. In ac elit eget ipsum pharetra faucibus. Maecenas viverra nulla in massa.

2.5.2 Localidade de referência

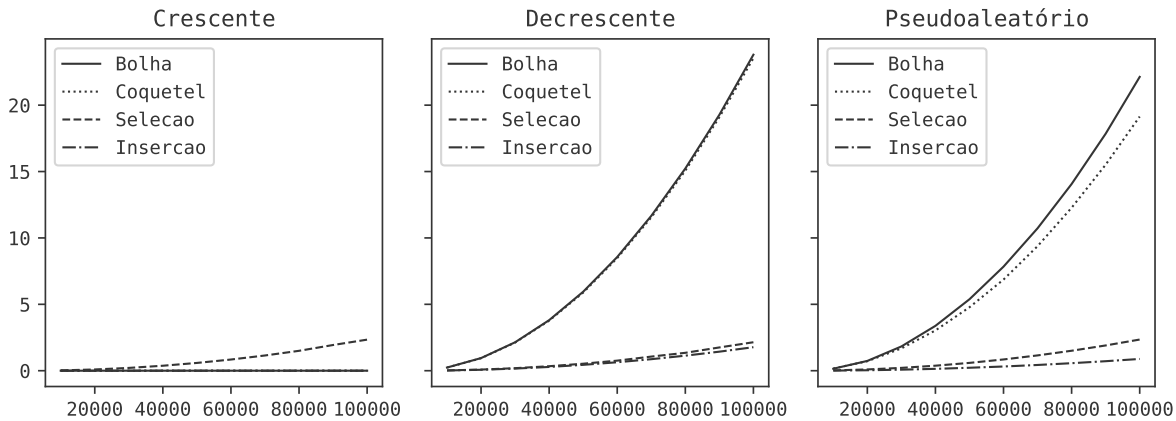
Nulla non mauris vitae wisi posuere convallis. Sed eu nulla nec eros scelerisque pharetra. Nullam varius. Etiam dignissim elementum metus. Vestibulum faucibus, metus sit amet mattis rhoncus, sapien dui laoreet odio, nec ultricies nibh augue a enim. Fusce in ligula. Quisque at magna et nulla commodo consequat. Proin accumsan imperdiet sem. Nunc porta. Donec feugiat mi at justo. Phasellus facilisis ipsum quis ante. In ac elit eget ipsum pharetra faucibus. Maecenas viverra nulla in massa.

3 ANÁLISE EMPÍRICA

Este capítulo tem como objetivo analisar empiricamente todos os algoritmos de ordenação apesentados anteriormente, comparando a análise teórica com os resultados obtidos na prática, no que diz respeito ao tempo de execução, número de comparações e número de movimentações.

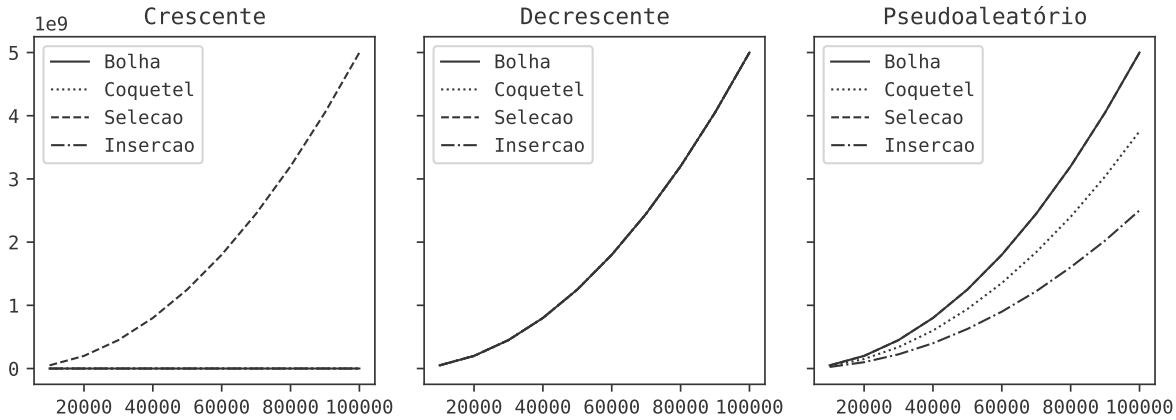
3.1 Métodos inferiores

Figura 1 – Métodos inferiores - Tamanho × Tempo.



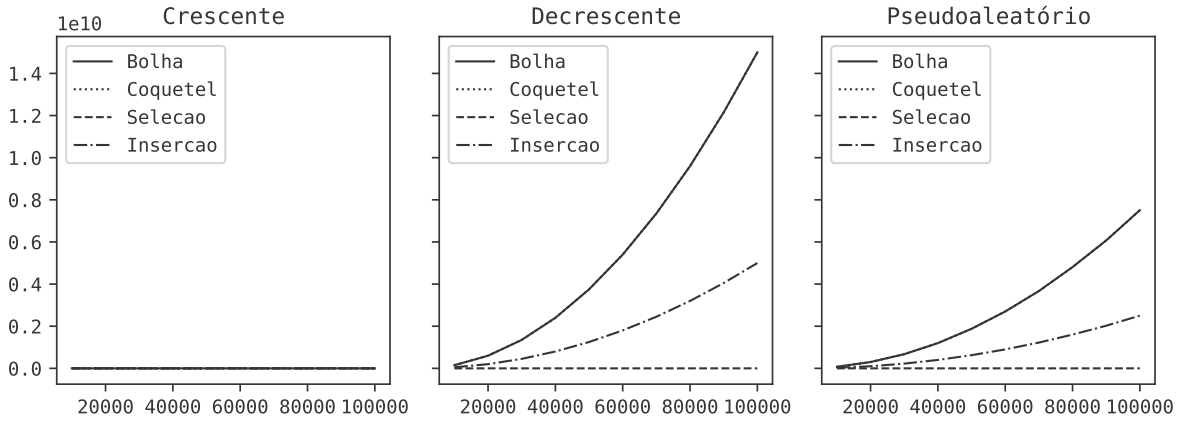
Fonte: Elaborado pelo autor

Figura 2 – Métodos inferiores - Tamanho × Comparações.



Fonte: Elaborado pelo autor

Figura 3 – Métodos inferiores - Tamanho × Movimentações.



Fonte: Elaborado pelo autor

REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 3rd. ed. Cambridge, MA: MIT Press, 2009. ISBN 978-0-262-03384-8.

KNUTH, D. E. **The Art of Computer Programming, Volume 3: Sorting and Searching**. 2nd. ed. Reading, Massachusetts: Addison-Wesley, 1998. ISBN 978-0201896855.